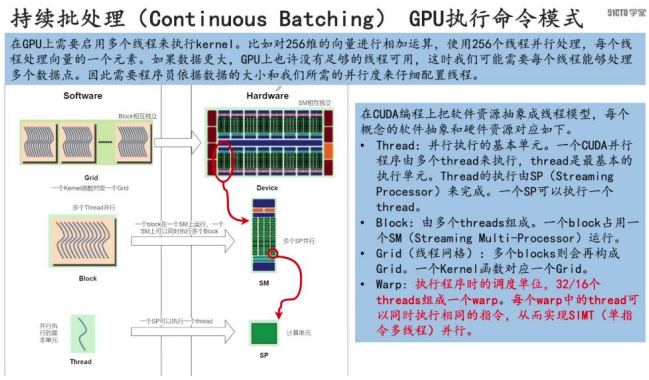
以下为AI生成的图文笔记的内容

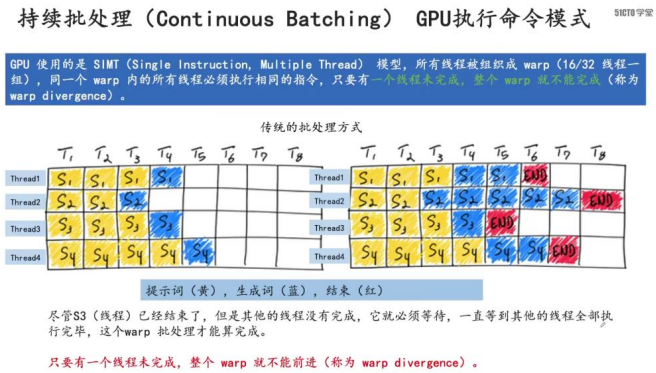
一、持续批处理 00:00

1. GPU执行命令模式



- **线程层级结构**:
  - **Thread**: 并行执行的基本单元，由SP(Streaming Processor)执行
  - **Block**: 由多个threads组成，占用一个SM(Streaming Multi-Processor)运行
  - **Grid**: 多个blocks构成，一个Kernel函数对应一个Grid
  - **Warp**: 32/16个threads组成的调度单位，实现SIMT(单指令多线程)并行
- **硬件对应关系**:
  - 一个SP执行一个thread
  - 一个SM运行一个block，可同时执行多个block
  - 一个Grid对应整个Kernel函数
  - Device包含所有硬件资源

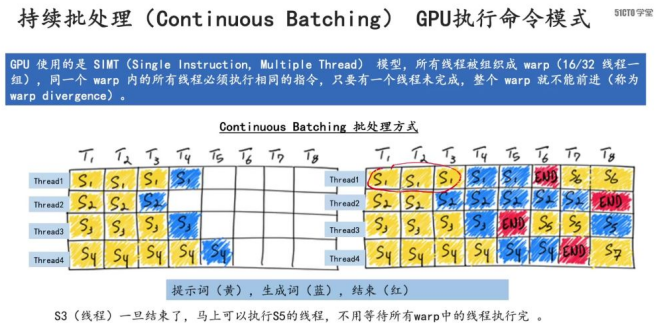
1）SIMT计算方式存在的问题 03:03



- 只要有一个线程未完成，整个 warp 就不能前进（称为 warp divergence）。
- **Warp同步机制**:
  - 同一warp内所有线程必须执行相同指令
  - 只要有一个线程未完成，整个warp就不能前进(warp divergence)
  - 即使部分线程提前完成，其SP资源也不能释放
- **实际案例**:
  - 假设一个warp包含4个线程处理不同长度token序列
  - 线程1处理3输入+2输出(共5个时间周期)
  - 线程2处理2输入+5输出(共7个时间周期)
  - 线程3和4较早完成，但仍需等待线程2完成才能释放资源
- **内存问题**:
  - 不同请求的KV Cache大小和生命周期不一致
  - 导致显存碎片化：部分缓存提前释放，部分扩展不足

- 显存利用率显著下降

2）持续批处理解决方案 06:10



-
- **核心改进**:
  - 不再等待warp内所有线程完成
  - 线程完成后立即释放SP资源供新线程使用
  - 实现动态资源调度
- **技术实现**:
  - 采用PagedAttention技术
    - 将长序列KV Cache拆分为"页"(page)
    - 使用页表(page table)记录逻辑到物理映射
    - 请求完成或扩展时只需更新映射，无需重新分配连续显存
- **优势效果**:
  - 解决token长度不一致导致的资源浪费
  - 动态分配/回收显存页
  - 同时提升GPU内存和计算资源利用率
  - 示例中线程1在第6周期完成后，线程6可立即使用释放的SP资源

二、SIMT计算方式存在的问题 08:19

- **内存碎片问题**: 在SIMT(单指令多线程)计算方式下，由于每个请求的token长度不同、生成速度不同，导致批次中不同请求的KVCache(注意力缓存)大小和生命周期都不一致。这会产生大量内存碎片，部分缓存提前释放而其他缓存扩展不足，最终造成显存利用率暴跌。
- **资源浪费现象**: 有些缓存或SP资源可以提前释放，但当前机制无法智能回收，导致这些资源不能被其他计算单元及时利用。

三、持续批处理解决方案 08:47

- **动态线程管理**: 持续批处理方案能够动态增加或删除对应线程，及时释放资源给更需要运算的单元使用。
- **内存分页技术**:
  - **分块存储**: 将长序列的KV Cache拆分为若干"页"(page)，通过页表(page table)记录逻辑到物理的映射关系
  - **动态映射**: 当请求完成或扩展时，只需更新映射关系而无需重新分配连续显存
- **协同优势**:
  - **资源分配**: 持续批处理动态分配线程，PagedAttention动态分配内存空间
  - **映射机制**: 通过页表实现内存空间与物理存储的动态映射
  - **综合效益**: 两者结合可充分利用GPU内存和计算资源，显著提升显存利用率

四、知识小结

知识点	核心内容	技术难点	解决方案

GPU线程调度模式	通过grid-block-SP层级结构执行线程，block在流式多处理器运行	**线程组同步约束**（SIMT模式需等待整组线程完成）	持续批处理技术
SIMT执行特性	单指令多线程（32/16线程为一组），SP执行相同指令	线程组内任意线程未完成则资源无法释放	动态资源释放机制
传统批处理瓶颈	不同请求的token长度差异导致KV缓存碎片化	**长尾请求阻塞资源**（如图中thread2阻塞thread1/3/4）	
持续批处理原理	实时释放已完成线程的SP资源，允许新线程插入	实现动态线程增减	与PagedAttention内存优化协同
技术协同效应	持续批处理优化计算资源，PagedAttention优化显存资源	计算/内存资源利用率双重提升	